

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Студент гр. 4343

И.И. Иванов

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: **И.И. Иванов**

Группа: 4343

Тема практики: Генетические алгоритмы

Задание на практику:

Дан взвешенный неориентированный граф (ребра имеют вес больше 0).
Необходимо найти минимальное остовное дерево для данного графа,
используя генетические алгоритмы.

Сроки прохождения практики: 06.07.2026 – 17.07.2026

Дата сдачи отчета: **00.00.0000**

Дата защиты отчета: **00.00.0000**

Студент

И.И. Иванов

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

В отчете представлены результаты выполнения учебной практики, посвященной исследованию и реализации генетического алгоритма для решения задачи поиска минимального остовного дерева взвешенного неориентированного графа. Он содержит описание предметной области, постановку задачи, результаты разработки, сведения об использованных технологиях и выводы по итогам выполненной работы.

SUMMARY

This report presents the results of an academic project focused on researching and implementing a genetic algorithm to solve the problem of finding the minimum spanning tree of a weighted undirected graph. It includes a description of the problem domain, a problem statement, development results, details regarding the technologies used, and conclusions based on the work performed.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	8
1.1 Предметная область	8
1.2 Задачи практики	8
1.2.1 Изучение предметной области	8
1.2.2 Анализ требований к программному средству.....	9
1.2.3 Проектирование структуры программного решения	9
1.2.4 Проектирование генетического алгоритма.....	9
1.2.5 Реализация алгоритма	9
1.2.6 Разработка графического пользовательского интерфейса.....	9
1.2.7 Реализация способов задания исходных данных	10
1.2.8 Тестирование программного средства	10
1.2.9 Анализ результатов	10
2 РЕЗУЛЬТАТЫ РАБОТЫ	11
2.1 Проектирование представления входных данных и генерации случайного графа11	
2.1.1 Формат хранения данных	11
2.2.1 Генерация случайного графа	11
2.2 Проектирование программного решения для генетического алгоритма ..	13
2.2.1 Представление особи	13
2.2.2 Функция исправления решения.....	13
2.2.3 Функция приспособленности	14
2.2.4 Турнирный отбор	15
2.2.5 Оператор скрещивания	15
2.2.6 Оператор мутации	16
2.2.7 Критерий завершения алгоритма	16
2.3 Проектирование графического интерфейса.....	17
2.4 Проектирование интеграции основного алгоритма в графический интерфейс	21
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	23
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	24

ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26
ПРИЛОЖЕНИЕ А	27

ВВЕДЕНИЕ

Предметной областью учебной практики являются методы решения задач комбинаторной оптимизации с использованием генетических алгоритмов, которые моделируют процессы естественного отбора и наследования для поиска оптимальных или близких к оптимальным решений. В рамках практики рассматривается задача поиска минимального остовного дерева взвешенного неориентированного графа, позволяющая продемонстрировать применение таких алгоритмов при решении сложных оптимизационных задач.

Целью учебной практики является разработка программного средства, реализующего поиск минимального остовного дерева с использованием генетического алгоритма, а также исследование особенностей его работы и визуализация процесса поиска решения.

Для достижения поставленной цели необходимо решить следующие задачи:

- Изучить теоретические основы генетических алгоритмов и особенности их применения при решении задач комбинаторной оптимизации;
- Изучить методы поиска минимального остовного дерева и определить особенности применения генетического алгоритма для решения данной задачи;
- Разработать способ представления графа и кодирования решения в виде хромосомы;
- Определить функцию приспособленности и выбрать генетические операторы отбора, скрещивания и мутации;
- Разработать архитектуру программного средства и определить взаимодействие основных программных модулей;
- Реализовать алгоритм поиска минимального остовного дерева с использованием генетического алгоритма;

- Разработать графический пользовательский интерфейс, обеспечивающий ввод исходных данных, настройку параметров алгоритма, управление процессом вычислений и визуализацию промежуточных и конечных результатов;
- Реализовать возможность задания исходных данных посредством ручного ввода, чтения из файла и случайной генерации графа;
- Выполнить тестирование разработанного программного средства на различных наборах данных и провести анализ полученных результатов.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Одной из важных задач теории графов является поиск минимального остовного дерева взвешенного графа. Остовным деревом называется подграф, содержащий все вершины исходного графа, не имеющий циклов и состоящий из минимально необходимого количества ребер. Минимальным остовным деревом является такое остовное дерево, суммарный вес ребер которого минимален среди всех возможных остовных деревьев данного графа.

Традиционно данная задача эффективно решается классическими алгоритмами, такими как алгоритмы Краскала и Прима. Однако использование генетических алгоритмов позволяет рассмотреть задачу с точки зрения методов оптимизации, основанных на моделировании процессов естественного отбора, наследования и мутации.

В рамках данной практики рассматривается реализация генетического алгоритма поиска минимального остовного дерева взвешенного неориентированного графа. Алгоритм должен обеспечивать формирование начальной популяции решений, вычисление функции приспособленности, выполнение операций отбора, скрещивания и мутации, а также последовательное улучшение найденных решений до достижения критерия остановки. Дополнительно предусматривается визуализация процесса работы алгоритма и анализ изменения качества решений по поколениям.

1.2 Задачи практики

1.2.1 Изучение предметной области

Изучить особенности задачи поиска минимального остовного дерева взвешенного неориентированного графа, рассмотреть существующие алгоритмы ее решения, а также исследовать принципы работы генетических алгоритмов и возможность их применения к задачам комбинаторной оптимизации.

1.2.2 Анализ требований к программному средству

Проанализировать требования, предъявляемые к разрабатываемому программному обеспечению, определить способы задания исходных данных, параметры настройки генетического алгоритма, требования к визуализации процесса поиска решения и отображению результатов работы.

1.2.3 Проектирование структуры программного решения

Разработать архитектуру программного средства, определить основные классы, структуры данных и взаимодействие отдельных модулей, отвечающих за представление графа, выполнение генетического алгоритма и взаимодействие с графическим интерфейсом.

1.2.4 Проектирование генетического алгоритма

Разработать структуру генетического алгоритма поиска минимального остова дерева, определить способ кодирования решений, выбрать функцию приспособленности, методы отбора, скрещивания и мутации, а также установить критерий завершения работы алгоритма.

1.2.5 Реализация алгоритма

Разработать программный модуль, реализующий генерацию начальной популяции, вычисление функции приспособленности, выполнение генетических операторов, формирование новых поколений и поиск наилучшего решения.

1.2.6 Разработка графического пользовательского интерфейса

Разработать графический пользовательский интерфейс, обеспечивающий удобное взаимодействие пользователя с программой. Интерфейс должен предоставлять возможность создания и редактирования графа, загрузки данных из файла, случайной генерации входных данных, настройки параметров генетического алгоритма, запуска и пошагового выполнения вычислений, отображения текущего состояния алгоритма, найденного остова дерева, а также графика изменения функции качества по поколениям.

1.2.7 Реализация способов задания исходных данных

Обеспечить возможность задания исходного графа несколькими способами: ручным вводом через графический интерфейс, чтением из файла и случайной генерацией графа.

1.2.8 Тестирование программного средства

Провести тестирование разработанного программного обеспечения на различных входных данных, проверить корректность работы генетического алгоритма и графического интерфейса.

1.2.9 Анализ результатов

Проанализировать результаты работы разработанного программного средства, оценить эффективность выбранных методов и сделать выводы о возможности применения генетического алгоритма для решения задачи поиска минимального остовного дерева.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1 Проектирование представления входных данных и генерации случайного графа

2.1.1 Формат хранения данных

Для хранения исходного графа выбрана структура данных, основанная на списке ребер. Такое представление является универсальным, компактным и удобно используется как при чтении графа из файла, так и при случайной генерации или вводе пользователем через графический интерфейс.

Каждое ребро графа содержит номера двух соединяемых вершин и вес ребра. В программной реализации ребро описывается структурой:

```
struct Edge
{
    int from;
    int to;
    double weight;
};
```

Полный граф представляет собой количество вершин и список всех ребер:

```
struct Graph
{
    int vertexCount;
    std::vector<Edge> edges;
};
```

Данный способ хранения позволяет быстро получать информацию о каждом ребре и использовать его индекс при построении хромосом генетического алгоритма.

Исходные данные также могут храниться в текстовом файле. В первой строке располагается количество вершин и количество ребер, после чего перечисляются все ребра графа в формате: *u v weight*. Такой формат является простым для чтения, легко редактируется вручную и удобен для тестирования программы.

2.2.1 Генерация случайного графа

Для обеспечения возможности тестирования генетического алгоритма без ручного ввода данных была реализована функция случайной генерации связного взвешенного неориентированного графа. Основной задачей

генератора является создание корректного графа, который удовлетворяет требованиям задачи поиска минимального остовного дерева.

Перед началом генерации выполняется проверка входных параметров. Если количество вершин меньше двух, оно автоматически увеличивается до минимально допустимого значения. Также контролируется число дополнительных ребер, которое не может быть отрицательным. После этого вычисляется максимально возможное количество ребер для простого неориентированного графа по формуле $m_{max} = \frac{n(n-1)}{2}$, где n – количество вершин графа. Если пользователь указал слишком большое количество дополнительных ребер, их число автоматически уменьшается до максимально допустимого значения.

Генерация графа выполняется в два этапа:

- На первом этапе строится связное остовное дерево, которое гарантирует связность итогового графа. Для этого вершины последовательно соединяются ребрами вида $(0, 1), (1, 2), \dots, (n - 2, n - 1)$. Каждому ребру случайным образом назначается вес из диапазона от 1 до 100. Благодаря этому уже после первого этапа граф является связным и содержит минимально необходимое количество ребер, равное $n - 1$.
- На втором этапе в граф добавляются дополнительные случайные ребра. Для исключения появления кратных ребер используется контейнер `std::set`, в котором хранятся все уже существующие пары вершин. Перед добавлением нового ребра проверяется выполнение следующих условий: вершины ребра различны, такое ребро еще отсутствует в графе, не превышено максимально допустимое количество ребер. Если хотя бы одно из условий нарушается, выполняется генерация новой пары вершин. Для предотвращения бесконечного цикла введено ограничение на количество попыток генерации новых ребер. Каждому успешно добавленному ребру также присваивается случайный вес из диапазона от 1 до 100.

Для генерации случайных чисел используется современный генератор `std::mt19937`, инициализируемый с помощью `std::random_device`. Такой подход обеспечивает хорошее качество случайного распределения и исключает повторяемость последовательностей при каждом запуске программы. Веса ребер генерируются равномерно, что позволяет получать разнообразные тестовые графы без выраженного смещения в сторону определенных значений.

2.2 Проектирование программного решения для генетического алгоритма

2.2.1 Представление особи

В данной работе каждая особь представляет собой список индексов ребер исходного графа без дубликатов. Таким образом, каждая хромосома определяет набор ребер, который рассматривается как кандидат на минимальное остовное дерево. Набор имеет длину $n - 1$, где n – количество вершин в графе.

Например, если граф содержит девять ребер и шесть вершин с индексами $\{0,1,2,3,4,5,6,7,8\}$, то особь $\{0,3,5,7,8\}$ означает, что предполагаемое дерево состоит именно из этих пяти ребер.

Такой способ кодирования обладает несколькими преимуществами: простотой реализации, небольшим объемом памяти, удобством выполнения генетических операторов, независимостью от конкретного расположения вершин графа.

2.2.2 Функция исправления решения

Недостатком реализованного представления особи является возможность появления недопустимых решений. После операций скрещивания или мутации выбранный набор ребер может содержать циклы, не соединять все вершины графа либо иметь дубликаты. Для устранения подобных ситуаций используется специальная процедура исправления решения.

Назначение процедуры *repair* – преобразование произвольного набора ребер в допустимое остовное дерево. Исправление выполняется в три последовательных этапа, каждый из которых устраняет свой тип нарушений:

- Устранение дубликатов. Набор ребер особи может содержать повторяющиеся индексы. Для их удаления производится сортировка списка индексов, после чего последовательные дубликаты исключаются с помощью стандартного алгоритма *unique*. В результате каждый индекс встречается не более одного раза.
- Исправление циклов. Для построения ациклического графа используется структура данных для непересекающихся множеств, инициализируемая для всех вершин графа. Ребра обрабатываются в том порядке, в котором они присутствуют в текущем наборе особи. Если добавление очередного ребра не создает цикла, ребро сохраняется в результирующий список, в противном случае оно отбрасывается. Таким образом, формируется лес, содержащий максимально возможное подмножество исходных ребер, не содержащее циклов.
- Восстановление связности. Если после предыдущего этапа количество сохраненных ребер меньше $n - 1$, где n – количество вершин, граф не является связным. Для объединения образовавшихся компонент выполняется просмотр всех ребер исходного графа в порядке возрастания их весов. Каждое ребро добавляется в дерево, если оно соединяет две разные компоненты связности. Процесс продолжается до тех пор, пока не будет набрано ровно $n - 1$ ребер, что гарантирует получение связного остова.

2.2.3 Функция приспособленности

Качество каждой особи оценивается функцией приспособленности. Поскольку целью задачи является минимизация суммарного веса остовного дерева, в качестве оценки используется прямая зависимость между стоимостью дерева и его приспособленностью.

Пусть W – суммарный вес ребер дерева. Тогда функция приспособленности определяется следующим образом: $fitness = W$.

Таким образом, чем меньше общий вес найденного дерева, тем меньше значение функции приспособленности, а следовательно, тем выше вероятность участия данной особи в формировании следующего поколения.

2.2.4 Турнирный отбор

Для формирования родительских пар выбран турнирный отбор. Суть метода заключается в том, что из текущей популяции случайным образом выбирается небольшая группа особей, после чего в качестве родителя выбирается особь с наименьшим значением функции приспособленности.

Выбор данного метода обусловлен несколькими причинами:

- Простотой программной реализации;
- Отсутствием необходимости вычислять вероятности выбора каждой особи;
- Устойчивостью к различным диапазонам значений функции приспособленности. По сравнению с рулеточным отбором турнирный метод менее чувствителен к масштабу функции качества и обеспечивает стабильную работу алгоритма;
- Хорошим балансом между исследованием новых решений и сохранением лучших найденных особей.

2.2.5 Оператор скрещивания

Для получения новых решений используется равномерное скрещивание. Для каждого гена независимо определяется, от какого родителя он будет унаследован. При формировании первого потомка для каждого гена с одинаковой вероятностью выбирается значение первого или второго родителя. Второй потомок получает противоположный выбор.

Для оператора задается вероятность выполнения, если значение меньше, то два выбранных родителя сразу отправляются в новое поколение, если

значение больше, то происходит скрещивание и только их дети проходят дальше.

Основным преимуществом такого метода является возможность свободного комбинирования отдельных удачных ребер из различных решений. В отличие от одноточечного или двухточечного скрещивания отсутствует привязка к расположению генов внутри хромосомы, что особенно важно при решении задачи поиска минимального остовного дерева, где порядок расположения ребер не оказывает влияния на качество решения.

После завершения операции скрещивания выполняется процедура исправления решения.

2.2.6 Оператор мутации

Мутация предназначена для поддержания разнообразия популяции и предотвращения преждевременной сходимости алгоритма.

В разработанном алгоритме мутация выполняется следующим образом:

- Случайным образом удаляется одно ребро;
- Случайным образом выбирается новое ребро исходного графа и добавляется в хромосому;
- Выполняется процедура восстановления корректного дерева.

Такой подход позволяет исследовать новые области пространства поиска без существенного нарушения структуры найденных решений.

2.2.7 Критерий завершения алгоритма

Для завершения работы генетического алгоритма используется комбинированный критерий остановки.

Алгоритм прекращает работу при выполнении одного из двух условий:

- Достигнуто максимальное количество поколений;
- Лучшее найденное решение не улучшается в течение заданного количества поколений подряд.

Использование сразу двух критериев позволяет, с одной стороны, гарантировать завершение работы программы, а с другой – избежать выполнения лишних вычислений после фактической сходимости популяции.

2.3 Проектирование графического интерфейса

Для обеспечения удобного взаимодействия пользователя с разработанным генетическим алгоритмом был реализован графический интерфейс пользователя. Интерфейс позволяет выполнять настройку параметров алгоритма, формировать входные данные различными способами, запускать процесс поиска решения и наблюдать за ходом выполнения алгоритма в режиме реального времени.

Разработка графического интерфейса выполнена с использованием библиотеки Dear ImGui, предназначенной для создания современных пользовательских интерфейсов в приложениях на языке C++. В качестве системы создания окна и обработки пользовательского ввода использовалась библиотека GLFW, обеспечивающая работу приложения независимо от операционной системы. Для построения графиков изменения функции качества решения применялась библиотека ImPlot, предоставляющая средства визуализации числовых данных непосредственно в интерфейсе программы.

Графический интерфейс разделен на несколько логических блоков, каждый из которых отвечает за выполнение определенного этапа работы программы.

Блок настройки параметров алгоритма (см. Рисунок 1). В данном разделе пользователь задает основные параметры генетического алгоритма, влияющие на процесс поиска решения. К ним относятся:

- Размер популяции;
- Максимальное количество поколений;
- Вероятность выполнения скрещивания;
- Вероятность выполнения мутации;
- Размер турнирного отбора.

Наличие данных параметров позволяет исследовать влияние настроек алгоритма на качество найденного решения и скорость его получения.

Рисунок 1 – Блок настройки параметров алгоритма

Блок формирования исходного графа (см. Рисунок 2). Интерфейс предусматривает несколько способов формирования входных данных:

- Загрузка графа из текстового файла;
- Случайная генерация связного взвешенного графа;
- Ручной ввод ребер графа через матрицу смежности (см. Рисунок 3).

Рисунок 2 – Блок формирования входных данных

	0	1	2	3	4
0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0

Рисунок 3 – Блок ручного ввода для графа

Блок управления выполнением алгоритма (см. Рисунок 4). Данный раздел содержит элементы управления процессом работы генетического алгоритма. Пользователь может выполнить:

- Инициализацию алгоритма;
- Запуск алгоритма до получения конечного результата;
- Выполнение одной итерации алгоритма;
- Остановку процесса вычислений;
- Переход к окончательному решению без отображения промежуточных поколений.

Подобная организация позволяет как проводить пошаговое исследование работы алгоритма, так и получать результат в автоматическом режиме.

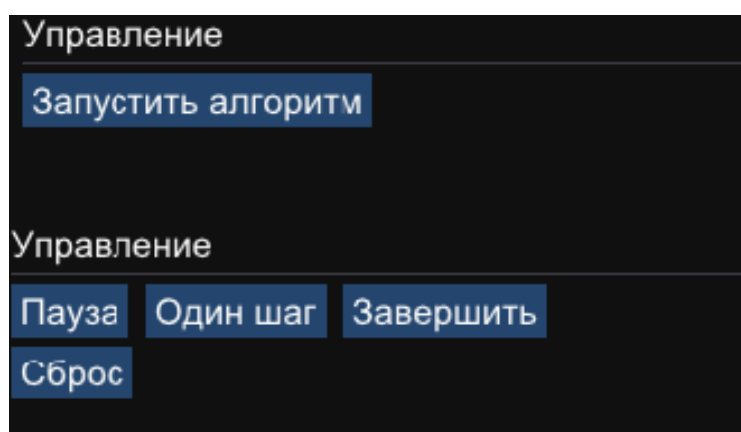


Рисунок 4 – Блок управления выполнением алгоритма

Блок визуализации графа (см. Рисунок 5). Одной из основных частей интерфейса является область отображения графа. В процессе работы программы отображаются вершины графа и соединяющие их ребра. Во время выполнения генетического алгоритма визуализируется текущее решение, соответствующее выбранной особи популяции или лучшему найденному основному дереву.

Визуализация позволяет наглядно проследить изменение структуры решения на протяжении выполнения алгоритма.

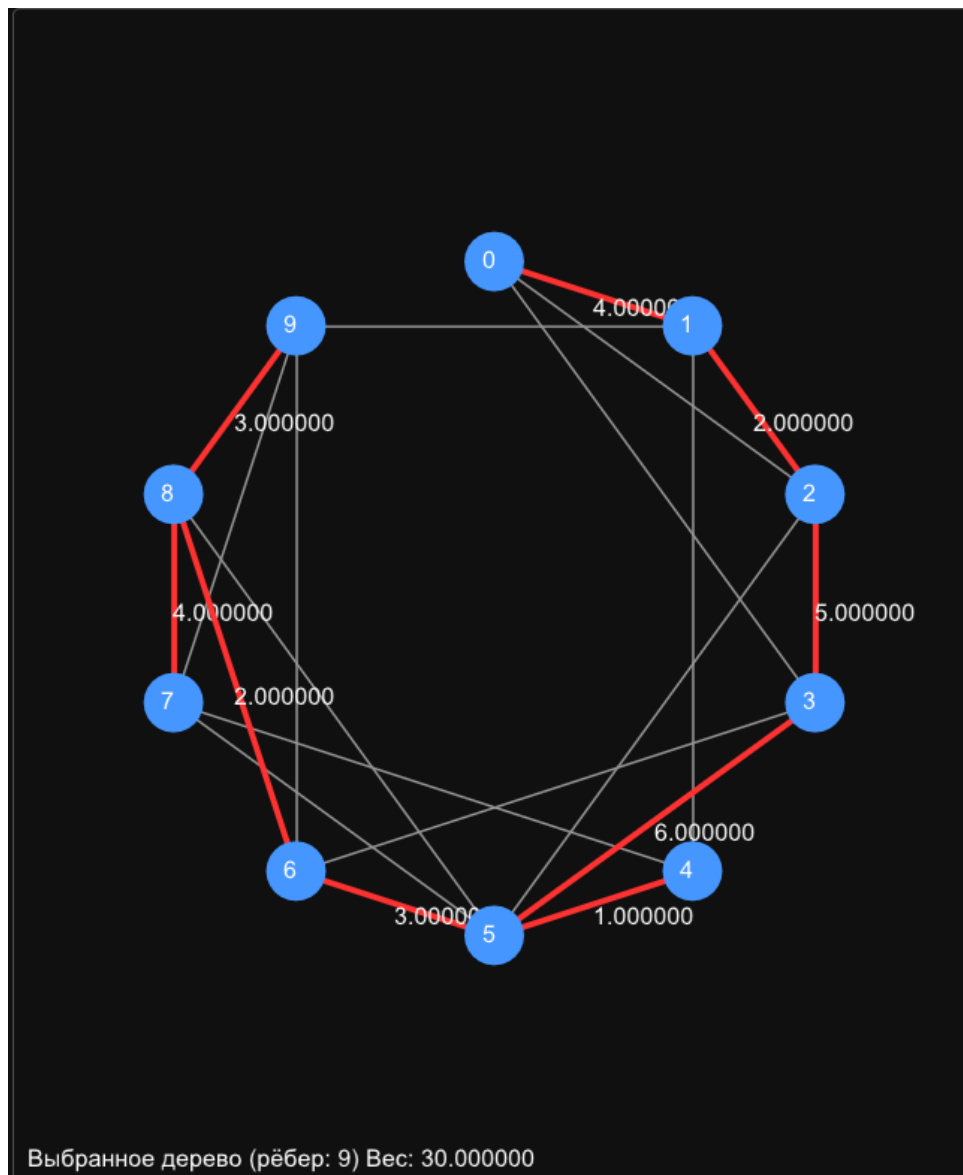


Рисунок 5 – Область визуализации графа

Блок отображения информации о популяции (см. Рисунок 6). Для анализа процесса работы генетического алгоритма предусмотрено отображение информации о текущем поколении и сформированной популяции. Пользователь может просматривать характеристики отдельных особей, значение функции приспособленности, суммарный вес найденного дерева и номер текущего поколения.

При необходимости осуществляется переключение между различными особями текущей популяции, что позволяет анализировать разнообразие решений на каждом этапе эволюции.

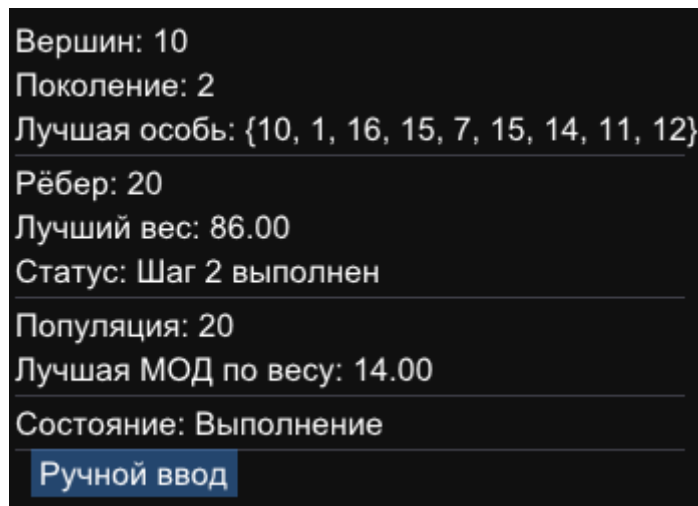


Рисунок 6 – Отображение текущей популяции

2.4 Проектирование интеграции основного алгоритма в графический интерфейс

Интеграция основного алгоритма в графический интерфейс реализована через разделение ответственности между модулями и организацию взаимодействия на основе глобальных переменных состояния. Основной генетический алгоритм инкапсулирован в классе `GeneticAlgorithm`, который предоставляет интерфейс для установки графа и параметров, инициализации популяции, выполнения одного поколения и получения текущих результатов. Графический интерфейс, реализованный с помощью библиотек `Dear ImGui`, `ImPlot` и `GLFW`, содержит отдельные панели для управления, отображения информации, визуализации графа и построения графиков эволюции.

Для синхронизации данных между интерфейсом и алгоритмом используется глобальный объект класса `GeneticAlgorithm`, доступный из любого модуля, что позволяет обработчикам кнопок и функциям отрисовки напрямую вызывать методы алгоритма и обновлять отображаемые значения.

При загрузке графа из файла, случайной генерации или ручном вводе производится инициализация алгоритма: объекту передается текущий граф, параметры, заданные пользователем (размер популяции, размер турнира, вероятности скрещивания и мутации, максимальное число поколений), после чего вызывается метод `initialize()`, формирующий начальную популяцию. С

этого момента в глобальных переменных сохраняются номер поколения, лучший вес и фитнес, история лучших значений для графика и набор ребер для подсветки.

Управление выполнением реализовано через кнопки: «Запустить алгоритм» запускает цикл, в котором последовательно выполняются шаги до достижения критерия остановки; после завершения все результаты обновляются в интерфейсе, включая график сходимости, который строится на основе вектора истории весов. Для визуализации текущего лучшего дерева используется массив индексов ребер, который передается в функцию отрисовки графа и отображается поверх основного графа жирными красными линиями.

Дополнительно предусмотрена возможность переключения между особями текущей популяции: при изменении номера особи в соответствующем поле ввода происходит обновление подсвечиваемого дерева и вывод информации о весе выбранной особи. Таким образом, интеграция обеспечивает полный цикл работы: от ввода данных до получения и визуализации результата, а также предоставляет пользователю гибкие средства контроля и анализа процесса оптимизации.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

4 ОТЗЫВ РУКОВОДИТЕЛЯ

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.

ПРИЛОЖЕНИЕ А